Maximum Subarray Sum of Size k

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

1. Problem Statement

Write a C program that reads the size and elements of a variable-length integer array arr[] and an integer k such that 0 < k < size. The array must be passed to a function named max_subarray_sum() using its address, and pointers must be used to pass the array between the calling function and max_subarray_sum().

The function max_subarray_sum() must:

- Identify all contiguous subarrays of size k in arr[].
- Calculate the sum of the elements in each subarray.
- Find and display the maximum subarray sum.
- Display -1 if k is invalid.

2. Objective

To understand and apply dynamic memory allocation, pointers, pointer arithmetic, passing arrays to functions by address, and nested loops in C by solving the fixed-size sliding window (maximum sum subarray) problem.

3. Concepts Used

- Pointers and pointer arithmetic: *(arr + i) is equivalent to arr[i].
- Dynamic memory allocation: malloc() creates an array whose size is decided at run time, and free() releases it.
- Passing an array to a function: the base address of the array is passed as an int * parameter.
- Nested for loops: the outer loop selects the starting index of each subarray, the inner loop adds its k elements.
- Input validation using if conditions.

4. Algorithm

1. Start.
2. Read the size of the array and allocate memory for it using malloc().
3. Read the size elements of the array using pointer notation (arr + i).
4. Read the value of k.
5. Call max_subarray_sum(arr, size, k), passing the base address of the array.
6. In the function, if k <= 0 or k >= size, print -1 and return.
7. For each starting index i from 0 to size - k: set sum = 0, add the k elements *(arr + i + j) for j = 0 to k - 1, and display the subarray with its sum.
8. If this is the first subarray or sum is greater than max_sum, update max_sum = sum.
9. After the loop, display the maximum subarray sum.
10. Free the allocated memory and stop.

Time complexity: O(n × k). **Space complexity:** O(1) extra space (apart from the input array).

5. Source Code

File name: `max_subarray_sum.c`

```
#include <stdio.h>
#include <stdlib.h>

/* Finds and displays the maximum sum of any contiguous subarray of size k.
   The array is accessed only through the pointer 'arr'. */
void max_subarray_sum(int *arr, int size, int k)
{
    int i, j, sum, max_sum;

    /* Validate k: must satisfy 0 < k < size */
    if (k <= 0 || k >= size)
    {
        printf("-1\n");
        return;
    }

    printf("\nSubarrays of size %d and their sums:\n", k);

    max_sum = 0;
    for (i = 0; i <= size - k; i++)
    {
        sum = 0;
        printf("  [");
        for (j = 0; j < k; j++)
        {
            sum += *(arr + i + j);          /* pointer notation */
            printf("%d%s", *(arr + i + j), (j < k - 1) ? " " : "");
        }
        printf("] = %d\n", sum);

        if (i == 0 || sum > max_sum)  /* first window sets max */
            max_sum = sum;
    }

    printf("\nMaximum subarray sum: %d\n", max_sum);
}

int main(void)
{
    int size, k, i;
    int *arr;

    printf("Enter the size of the array: ");
    if (scanf("%d", &size) != 1 || size <= 0)
    {
        printf("-1\n");
        return 1;
    }

    arr = (int *)malloc(size * sizeof(int));   /* variable-length array */
    if (arr == NULL)
    {
        printf("Memory allocation failed.\n");
```

```
        return 1;
    }

    printf("Enter %d elements: ", size);
    for (i = 0; i < size; i++)
        scanf("%d", arr + i);

    printf("Enter the value of k: ");
    scanf("%d", &k);

    max_subarray_sum(arr, size, k);            /* pass address of array */

    free(arr);
    return 0;
}
```

6. Sample Input and Output

The program was compiled with gcc -Wall -Wextra and tested on the three examples from the problem statement and one extra case containing negative numbers. Input values typed by the user are shown after each prompt.

Test Case 1

```
Enter the size of the array: 6
Enter 6 elements: 2 1 5 1 3 2
Enter the value of k: 3

Subarrays of size 3 and their sums:
  [2 1 5] = 8
  [1 5 1] = 7
  [5 1 3] = 9
  [1 3 2] = 6

Maximum subarray sum: 9
```

Test Case 2

```
Enter the size of the array: 5
Enter 5 elements: 2 3 4 1 5
Enter the value of k: 2

Subarrays of size 2 and their sums:
  [2 3] = 5
  [3 4] = 7
  [4 1] = 5
  [1 5] = 6

Maximum subarray sum: 7
```

Test Case 3 (invalid k)

```
Enter the size of the array: 3
Enter 3 elements: 1 2 3
Enter the value of k: 4
-1
```

Test Case 4 (negative numbers)

```
Enter the size of the array: 4
Enter 4 elements: -3 -1 -2 -5
Enter the value of k: 2

Subarrays of size 2 and their sums:
  [-3 -1] = -4
  [-1 -2] = -3
  [-2 -5] = -7

Maximum subarray sum: -3
```

7. Explanation of Example 1

For the array 2 1 5 1 3 2 with k = 3, there are size - k + 1 = 4 subarrays of size 3: 2+1+5 = 8, 1+5+1 = 7, 5+1+3 = 9 and 1+3+2 = 6. The largest of these sums is 9, so the program prints Maximum subarray sum: 9.

For Example 3, k = 4 is greater than the array size 3, so the condition 0 < k < size fails and the function prints -1.

8. Conclusion

The program correctly finds the maximum sum of all contiguous subarrays of size k. The array is created dynamically, passed to max_subarray_sum() by its address, and accessed only through pointers. Invalid values of k are handled by displaying -1, and the results match all the given examples.